

GLIMA

Plataforma experimental para la adquisición, análisis, almacenamiento y visualización de variables hidrológicas y ambientales, desarrollada en el contexto del Grupo G-LIMA.

El proyecto integra principalmente dos instrumentos:

- el pluviómetro óptico **Hydreon RG-15**, para medir lluvia;
- el sensor ultrasónico **MaxBotix MB7388**, para medir distancia y apoyar aplicaciones de seguimiento de nivel.

Uno de los objetivos principales, cumplido satisfactoriamente, fue integrar ambos sensores en una Raspberry Pi, procesar sus mediciones, generar un respaldo local en archivos CSV y TXT y transmitir la información a un dashboard en ThingsBoard.

Una segunda línea de desarrollo estudia el uso de módulos **XBee-PRO 900HP** como alternativa de comunicación inalámbrica entre los sensores, una ESP32 y la Raspberry Pi. Esta alternativa no constituye el propósito final del proyecto, sino una forma de ampliar su aplicabilidad en estaciones remotas, puntos de medición separados del equipo de procesamiento o lugares con conectividad limitada.

Estado del proyecto: la adquisición, el análisis, el almacenamiento local y la visualización conjunta de lluvia y distancia ya fueron implementados. La comunicación XBee se encuentra implementada para el MB7388 y se plantea extenderla también al RG-15.

Contenido

- [Propósito del proyecto](#)
- [Resultados alcanzados](#)
- [Arquitecturas desarrolladas](#)
- [Hardware](#)
- [Variables procesadas](#)
- [Estructura del repositorio](#)
- [Programas principales](#)
- [Procesamiento de los datos](#)
- [Requisitos de software](#)
- [Instalación](#)
- [Configuración y ejecución](#)
- [Archivos generados](#)
- [ThingsBoard](#)
- [Comunicación XBee](#)
- [Solución de problemas](#)

- [Seguridad](#)
- [Documentación técnica](#)

Propósito del proyecto

El propósito general es desarrollar una plataforma modular para la gestión de datos hidrológicos, desde su adquisición en campo hasta su almacenamiento y visualización.

Los objetivos específicos son:

1. adquirir datos de lluvia con el RG-15;
2. adquirir datos de distancia con el MB7388;
3. validar, analizar y consolidar las mediciones en una Raspberry Pi;
4. calcular variables representativas e indicadores de calidad;
5. conservar los datos localmente en formatos legibles y estructurados;
6. transmitir la telemetría a un dashboard para su consulta remota;
7. mantener el registro local cuando no exista conexión a Internet;
8. evaluar protocolos alternativos, como XBee, para comunicar nodos separados o estaciones con conectividad limitada;
9. avanzar hacia una arquitectura inalámbrica que pueda transportar tanto los datos de lluvia como los de distancia.

Los primeros siete objetivos fueron implementados mediante la integración directa de los sensores con la Raspberry Pi. El enlace XBee corresponde a una ampliación de la plataforma y actualmente funciona con el sensor de distancia.

Resultados alcanzados

La plataforma desarrollada permite:

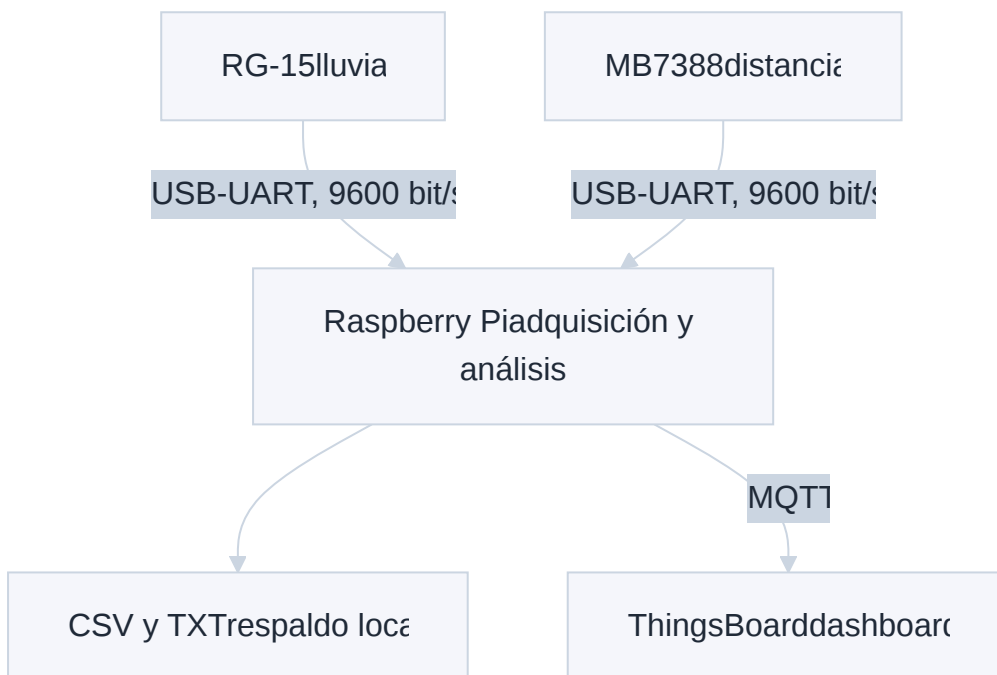
- consultar y procesar las mediciones del RG-15;
- recibir continuamente las tramas del MB7388;
- calcular la moda de las distancias medidas durante cada intervalo;
- conservar los acumulados de lluvia informados por el propio RG-15;
- calcular porcentajes de datos correctos para lluvia y distancia;
- registrar fecha y hora en la zona **America/Bogota**;
- crear archivos CSV para análisis posterior;
- crear archivos TXT tabulados para inspección directa;
- rotar automáticamente los archivos por hora, día, mes o en un archivo único;
- enviar telemetría a ThingsBoard Cloud mediante MQTT;
- continuar el almacenamiento local si la conexión con ThingsBoard falla;

- comunicar inalámbricamente las mediciones del MB7388 entre una ESP32 y la Raspberry Pi mediante XBee;
- detectar mensajes XBee repetidos, posibles pérdidas y reinicios mediante un número de secuencia.

Arquitecturas desarrolladas

1. Integración de lluvia y distancia en la Raspberry Pi

Esta arquitectura cumplió el objetivo de recibir, analizar, guardar y visualizar conjuntamente los datos del RG-15 y del MB7388.

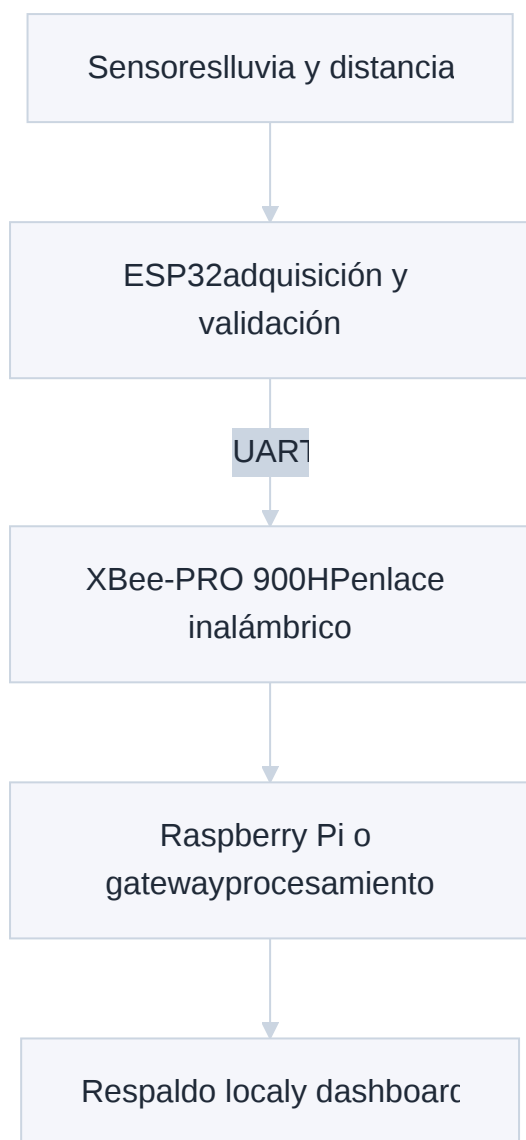


El RG-15 y el MB7388 se conectan a la Raspberry Pi mediante adaptadores USB-UART. La Raspberry procesa ambos flujos, genera los archivos locales y publica la telemetría.

El acceso a Internet solo es necesario para actualizar ThingsBoard. Si la conexión falla, el programa puede conservar los datos localmente.

2. Comunicación inalámbrica mediante XBee

Esta arquitectura busca separar físicamente el punto de medición del equipo encargado del almacenamiento y la visualización.



En la implementación disponible, el MB7388 se conecta a la ESP32 y sus datos se transmiten mediante XBee a la Raspberry Pi. El propósito de esta línea es ampliar posteriormente el protocolo para transportar también los datos del RG-15.

XBee proporciona un enlace local que no depende de Internet para trasladar las mediciones entre nodos. Para transmitir los datos a un dashboard remoto seguirá siendo necesario que la Raspberry Pi, otro gateway o un nodo posterior disponga de Internet. Si ningún nodo tiene conectividad, los datos pueden conservarse localmente hasta que sea posible sincronizarlos.

Hardware

Componente	Función
Raspberry Pi 4 Model B	Procesamiento, almacenamiento local y transmisión al dashboard
Hydreon RG-15	Medición óptica de lluvia

Componente	Función
MaxBotix MB7388 HRXL-MaxSonar-WR	Medición ultrasónica de distancia
Adaptadores USB-UART CP2102	Conexión serial directa de los sensores a la Raspberry Pi
ESP32 DevKit V1 / ESP-WROOM-32	Adquisición y validación en la arquitectura XBee
2 × XBee-PRO 900HP	Enlace inalámbrico entre la ESP32 y la Raspberry Pi o gateway
Adaptador XBee USB	Conexión del XBee receptor a la Raspberry Pi
Fuentes reguladas y cableado	Alimentación estable y tierra común

Los puertos seriales utilizados por los programas trabajan a **9600 bit/s, 8 bits de datos, sin paridad y un bit de parada**.

Variables procesadas

Lluvia

El RG-15 entrega una respuesta como:

```
Acc 0.00 mm, EventAcc 0.00 mm, TotalAcc 0.00 mm, RInt 0.00 mmph
```

De esta respuesta se extraen:

Variable	Descripción
lluvia_adicional_mm	Lluvia adicional registrada en la consulta
lluvia_evento_mm	Acumulado del evento de lluvia
lluvia_total_mm	Acumulado total mantenido por el RG-15
intensidad_lluvia_mm_h	Intensidad de lluvia en mm/h

El programa utiliza directamente el valor **TotalAcc** mantenido por el sensor; no reconstruye el acumulado sumando mediciones en la Raspberry Pi.

Distancia

El MB7388 transmite tramas con el formato:

R####

Ejemplos:

R1340

R9999

Una distancia entre 0 y 9998 mm se considera numéricamente válida. El valor **R9999** indica que el sensor respondió, pero no detectó un objetivo.

Durante cada intervalo de guardado, la Raspberry conserva las distancias válidas y calcula la moda. Si varias distancias tienen la misma frecuencia máxima, selecciona la que apareció más recientemente.

Calidad de los datos

La integración conjunta calcula dos indicadores:

- **porcentaje_datos_correctos_lluvia;**
- **porcentaje_datos_correctos_distancia.**

Los porcentajes son acumulativos dentro del archivo vigente. Con la configuración horaria, se reinician al comenzar cada archivo nuevo.

Para distancia, una moda entre 0 y 9998 mm cuenta como correcta. Un valor 9999 o una fila sin distancia disminuye el porcentaje.

En el enlace XBee, el número de secuencia permite diagnosticar paquetes repetidos o posiblemente perdidos. Este diagnóstico es independiente del porcentaje de calidad de las filas consolidadas.

Estructura del repositorio

```
GLIMA/
├── Datasheets/
│   ├── CP2102.PDF
│   ├── ESP32.PDF
│   ├── Esp32_DevKitC_Pinoutpng.png
│   ├── HRXL-MaxSonar-WR_Datasheet.pdf
│   ├── XBee-PRO900HP_DigiMesh_KitUserGuide.pdf
│   ├── Xbee_pinout.pdf
│   └── rg-15_instructions.pdf
├── PlatformIO_ESP32_DOIT_DEVKITC/
│   ├── Sensores_Esp32/
│   │   ├── Prueba_RG-15/
│   │   ├── Prueba_SensorMB7388/
│   │   └── SensoresIntegrados_v1/
│   ├── Xbee_Esp32/
│   │   └── Xbee_Esp32_2_Rasp_v1/
└── Raspberrypi_4_ModelB/
    ├── Sensores_Rasp/
    └── Xbee_Rasp/
```

Ruta	Contenido
Datasheets	Manuales, hojas de datos y diagramas de pines
PlatformIO_ESP32_DOIT_DEVKITC/Sensores_Esp32	Pruebas individuales e integración de sensores en ESP32
PlatformIO_ESP32_DOIT_DEVKITC/Xbee_Esp32	Desarrollo del enlace MB7388–ESP32–XBee–Raspberry Pi
Raspberrypi_4_ModelB/Sensores_Rasp	Adquisición directa, integración de lluvia y distancia, almacenamiento y dashboard
Raspberrypi_4_ModelB/Xbee_Rasp	Recepción de datos transmitidos mediante XBee

Programas principales

Integración completa de lluvia y distancia

Ruta:

```
Raspberrypi_4_ModelB/Sensores_Rasp/
sensores_v4_dashboard.py
```

Este programa representa la integración que cumplió el objetivo de:

- recibir los dos sensores en la Raspberry Pi;
- procesar lluvia y distancia;
- calcular indicadores de calidad;
- guardar los datos en CSV y TXT;
- enviar la telemetría a ThingsBoard.

Configuración inicial:

Parámetro	Valor
Puerto RG-15	/dev/ttyUSB0
Puerto MB7388	/dev/ttyUSB1
Velocidad serial	9600 bit/s
Consulta del RG-15	60 segundos
Guardado y telemetría	30 segundos
Rotación de archivos	Por hora
Zona horaria	America/Bogota

Los nombres de los puertos deben verificarse en cada Raspberry Pi, pues pueden cambiar al reconectar los adaptadores.

Enlace XBee para distancia

ESP32:

```
PlatformIO_ESP32_DOIT_DEVKITC/Xbee_Esp32/
Xbee_Esp32_2_Rasp_v1/
```

El archivo **src/main.cpp** recibe el MB7388 por UART2, valida sus tramas, añade un número de secuencia y transmite la medición por UART1 hacia el XBee.

Raspberry Pi:

```
Raspberrypi_4_ModelB/Xbee_Rasp/
distancia_xbee_esp32_v2_dashb_plus_csv.py
```

Este receptor valida los mensajes XBee, calcula la moda cada 30 segundos, guarda CSV y TXT y envía distancia y calidad a ThingsBoard.

El archivo **distancia_xbee_esp32.py** corresponde a una versión más sencilla para recepción y pruebas del enlace.

Otros desarrollos

Los demás programas documentan etapas del proyecto, pruebas individuales y versiones previas. No son simplemente archivos obsoletos: permiten revisar la evolución de la lectura de cada sensor, su integración y las distintas estrategias de almacenamiento y visualización.

Procesamiento de los datos

Integración directa

El programa **sensores_v4_dashboard.py** realiza de forma no bloqueante las siguientes tareas:

1. consulta periódicamente el RG-15;
2. recibe y valida sus respuestas;
3. recibe continuamente las tramas del MB7388;
4. acumula las distancias durante el intervalo;
5. calcula la moda de distancia;
6. actualiza los indicadores de calidad;
7. escribe una fila en CSV y TXT;
8. envía la misma fila a ThingsBoard;
9. inicia un nuevo intervalo de recolección.

Enlace XBee

La ESP32 transmite:

```
D,secuencia,distancia_mm,estado
```

Ejemplo:

```
D,25,1340,0
```

Estado	Distancia	Significado
0	0 a 9998	Distancia válida
1	9999	Sensor activo, sin objetivo detectado
2	-1	No llegó una trama R#### válida

La configuración actual de la ESP32 transmite un mensaje cada 2 segundos:

```
FRECUENCIA_ENVIO_MS = 2000
```

La Raspberry consolida los mensajes recibidos durante 30 segundos antes de guardar y publicar una fila.

Requisitos de software

Computador de desarrollo

- Visual Studio Code;
- extensión PlatformIO IDE;
- plataforma Espressif 32;
- framework Arduino para ESP32;
- controlador USB correspondiente a la placa.

Raspberry Pi

- Raspberry Pi OS;
- Python 3.9 o posterior;
- paquetes **pyserial** y **tb-mqtt-client**;
- acceso del usuario a los puertos seriales;
- Internet únicamente cuando se requiera transmitir a ThingsBoard.

Instalación

1. Clonar el repositorio

```
git clone https://github.com/tpalaciof/GLIMA.git
cd GLIMA
```

2. Preparar Python en la Raspberry Pi

Se recomienda crear un entorno virtual:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install pyserial tb-mqtt-client
```

Utilice **python -m pip** dentro del entorno activo. Así las dependencias se instalan en el mismo intérprete que ejecutará el programa y se evitan conflictos con **pyserial**.

Si el usuario no tiene permiso para abrir los puertos seriales:

```
sudo usermod -aG dialout $USER
```

Cierre la sesión y vuelva a entrar para aplicar el cambio.

3. Identificar los puertos

```
ls -l /dev/ttyUSB*  
python3 -m serial.tools.list_ports
```

Para la integración directa se necesitan normalmente dos puertos: uno para el RG-15 y otro para el MB7388. Para el receptor XBee se necesita el puerto del adaptador XBee USB.

Configuración y ejecución

Opción A: lluvia y distancia conectadas a la Raspberry Pi

Entre a:

```
cd Raspberrypi_4_ModelB/Sensores_Rasp
```

Revise en **sensores_v4_dashboard.py**:

```
PUERTO_RG15 = "/dev/ttyUSB0"  
PUERTO_MB7388 = "/dev/ttyUSB1"  
ACCESS_TOKEN = "TOKEN_PRIVADO"  
THINGSBOARD_SERVER = "mqtt.thingsboard.cloud"
```

Ejecute:

```
python sensores_v4_dashboard.py
```

Esta es la opción que integra los dos sensores, genera archivos conjuntos y publica todas las variables en el dashboard.

Opción B: distancia transmitida mediante XBee

Conexiones MB7388-ESP32

MB7388	ESP32
TX serial	GPIO16, RX de UART2
GND	GND
Alimentación	Fuente apropiada según la hoja de datos

Conexiones XBee-ESP32

XBee	ESP32
DOUT	GPIO26, RX de UART1
DIN	GPIO27, TX de UART1
GND	GND
VCC	Fuente regulada apropiada

No alimente el XBee desde un pin GPIO. Utilice una placa adaptadora o una fuente regulada capaz de suministrar la corriente requerida.

Abra en PlatformIO:

```
PlatformIO_ESP32_D0IT_DEVKITC/Xbee_Esp32/  
Xbee_Esp32_2_Rasp_v1
```

Compile, cargue y abra el monitor:

```
pio run  
pio run --target upload  
pio device monitor
```

El monitor USB trabaja a 115200 bit/s. El MB7388 y el XBee trabajan a 9600 bit/s.

En la Raspberry Pi:

```
cd Raspberrypi_4_ModelB/Xbee_Rasp
```

Revise en **distancia_xbee_esp32_v2_dashb_plus_csv.py**:

```
PUERTO_XBEE = "/dev/ttyUSB0"  
ACCESS_TOKEN = "TOKEN_PRIVADO"
```

Ejecute:

```
python distancia_xbee_esp32_v2_dashb_plus_csv.py
```

Archivos generados

Los programas crean automáticamente una carpeta:

```
datos_sensores/
```

Los TXT quedan directamente en esa carpeta y los CSV dentro de:

```
datos_sensores/csv/
```

La frecuencia de rotación se controla mediante:

```
FRECUENCIA_ARCHIVO = "hora"
```

Opciones disponibles:

- hora;
- diario;
- mensual;
- unico.

Integración conjunta

Nombre base:

```
mediciones
```

Columnas del CSV:

Columna	Descripción
fecha	Fecha local
hora	Hora local
lluvia_adicional_mm	Lluvia adicional
lluvia_evento_mm	Acumulado del evento
lluvia_total_mm	Acumulado total del RG-15
intensidad_lluvia_mm_h	Intensidad de lluvia
distancia_moda_mm	Moda de distancia
porcentaje_datos_correctos_lluvia	Calidad acumulativa de lluvia
porcentaje_datos_correctos_distancia	Calidad acumulativa de distancia

Enlace XBee de distancia

Nombre base:

```
mediciones_distancia
```

Columnas del CSV:

Columna	Descripción
fecha	Fecha local
hora	Hora local
distancia_moda_mm	Moda de distancia
porcentaje_datos_correctos_distancia	Calidad acumulativa de distancia

Los TXT contienen la misma información en tablas alineadas para facilitar la lectura humana.

ThingsBoard

ThingsBoard se utiliza como plataforma de visualización remota de las mediciones procesadas por la Raspberry Pi. Los programas se conectan a ThingsBoard Cloud mediante MQTT y publican la telemetría en:

```
mqtt.thingsboard.cloud
```

La Raspberry Pi se autentica como dispositivo mediante el valor configurado en **ACCESS_TOKEN**. Este token permite publicar telemetría y es diferente del correo y la contraseña utilizados por una persona para entrar a la interfaz web.

Flujo de la telemetría

Cada 30 segundos, el programa consolida los datos del intervalo, los guarda localmente y construye un mensaje con:

- una marca de tiempo;
- las variables disponibles;
- los indicadores de calidad correspondientes.

La estructura enviada a ThingsBoard es equivalente a:

```
{
  "ts": 1788447600000,
  "values": {
    "lluvia_total_mm": 12.45,
    "distancia_moda_mm": 1340,
    "porcentaje_datos_correctos_lluvia": 100.0,
    "porcentaje_datos_correctos_distancia": 96.67
  }
}
```

La propiedad **ts** corresponde al instante de la medición expresado en milisegundos desde Unix epoch. Se genera en la Raspberry Pi usando la zona horaria **America/Bogota**, de modo que ThingsBoard pueda ubicar correctamente cada dato en las series temporales.

Las variables cuyo valor sea inexistente se omiten del mensaje. Esto evita enviar valores nulos como si fueran mediciones reales.

Claves de la integración conjunta

El programa **sensores_v4_dashboard.py** puede publicar:

Clave	Unidad	Descripción	Visualización sugerida
lluvia_adicional_mm	mm	Lluvia adicional reportada en la consulta	Serie temporal o tarjeta
lluvia_evento_mm	mm	Acumulado del evento de lluvia	Serie temporal
lluvia_total_mm	mm	Acumulado total mantenido por el RG-15	Serie temporal o tarjeta
intensidad_lluvia_mm_h	mm/h	Intensidad instantánea de lluvia	Serie temporal o indicador
distancia_moda_mm	mm	Moda de las distancias válidas del intervalo	Serie temporal
porcentaje_datos_correctos_lluvia	%	Calidad acumulativa de los datos de lluvia	Indicador o medidor
porcentaje_datos_correctos_distancia	%	Calidad acumulativa de los datos de distancia	Indicador o medidor

El RG-15 se consulta cada 60 segundos, mientras que los datos se guardan y se publican cada 30 segundos. Por esta razón, los valores de lluvia pueden mantenerse iguales entre dos consultas consecutivas del sensor.

Claves del receptor XBee actual

El programa `distancia_xbee_esp32_v2_dashb_plus_csv.py` publica:

Clave	Unidad	Descripción	Visualización sugerida
distancia_moda_mm	mm	Moda de las distancias recibidas por XBee durante el intervalo	Serie temporal
porcentaje_datos_correctos_distancia	%	Calidad acumulativa de las filas de distancia	Indicador o medidor

Acceso y visualización del dashboard

Para consultar los datos:

- ingrese a [ThingsBoard Cloud](#);
- inicie sesión con las siguientes credenciales:

Correo: user123@mail.com
Contraseña: hola1234

- seleccione el dashboard asociado a la estación o al dispositivo;;
- revise las gráficas de lluvia y distancia y los indicadores de calidad.

Estas credenciales corresponden únicamente a una cuenta de demostración destinada a facilitar la visualización del proyecto.

Funcionamiento cuando ThingsBoard no está disponible

Los archivos CSV y TXT constituyen el respaldo local de las mediciones. El receptor XBee está preparado para que una falla de conexión con ThingsBoard no impida el guardado local y para intentar restablecer la comunicación posteriormente.

Los datos que no lleguen al dashboard deben verificarse en los archivos locales. La versión actual no realiza automáticamente una carga histórica de todas las filas que hayan quedado pendientes durante una interrupción; esa sincronización requeriría una función adicional.

Comunicación XBee

Los dos XBee deben:

- pertenecer a la misma red;
- utilizar parámetros DigiMesh compatibles;

- trabajar con los mismos parámetros seriales;
- disponer de alimentación estable;
- tener DOUT, DIN y GND correctamente conectados.

El enlace XBee resulta especialmente útil cuando:

- los sensores están alejados de la Raspberry Pi;
- no es conveniente tender cableado serial;
- el punto de medición no dispone de Internet;
- se requiere transportar la información por radio hasta un gateway;
- se desea mantener la adquisición independiente de la disponibilidad de la nube.

La ampliación prevista consiste en definir e implementar mensajes XBee para las variables del RG-15, de manera que la arquitectura inalámbrica pueda transportar lluvia y distancia.

Solución de problemas

No aparecen los puertos USB

```
lsusb
ls -l /dev/ttyUSB*
dmesg | tail -n 30
```

Desconecte y conecte cada adaptador por separado para identificarlo. Actualice los nombres de puerto en el programa.

Permission denied al abrir un puerto

```
groups
sudo usermod -aG dialout $USER
```

Después debe cerrar la sesión y volver a entrar.

No module named serial

Active el entorno correcto:

```
source .venv/bin/activate
python -m pip install pyserial
python -c "import serial; print(serial.__file__)"
```

No instale un paquete llamado únicamente **serial**; el módulo requerido pertenece a **pyserial**.

No se guardan los datos

- confirme que se está ejecutando el programa correcto;
- espere al menos un intervalo completo de 30 segundos;
- revise los mensajes de error de la terminal;
- compruebe permisos de escritura en la carpeta del programa;
- verifique que los puertos correspondan a cada dispositivo;
- confirme que el entorno virtual contiene las dependencias.

No aparecen datos en ThingsBoard

- confirme primero que los CSV y TXT se estén generando;
- revise la conexión a Internet;
- verifique el servidor MQTT;
- compruebe el token del dispositivo;
- revise que los widgets utilicen exactamente las claves de telemetría.

No llegan mensajes XBee

- compruebe la configuración de ambos módulos;
- confirme 9600 bit/s y 8N1;
- revise DOUT, DIN y GND;
- verifique la salida de la ESP32 en el monitor de PlatformIO;
- pruebe el XBee receptor con un monitor serial.

El MB7388 entrega R9999

R9999 no indica necesariamente una falla de comunicación. Significa que el sensor respondió correctamente, pero no detectó un objetivo.

Seguridad

- No publique tokens de ThingsBoard, contraseñas ni credenciales.
- Si una credencial fue incluida en un repositorio público, debe revocarse y reemplazarse.
- Para una versión de producción, lea el token desde una variable de entorno o un archivo local excluido mediante **.gitignore**.
- Utilice un token distinto para cada estación.
- Mantenga copias de respaldo de los datos y de la configuración de los XBee.

Documentación técnica

- [Hoja de datos del MB7388](#)

- [Manual del RG-15](#)
- [Guía del kit XBee-PRO 900HP DigiMesh](#)
- [Diagrama de pines XBee](#)
- [Hoja de datos de la ESP32](#)
- [Diagrama de pines ESP32 DevKitC](#)
- [Hoja de datos del CP2102](#)
- [SDK oficial de ThingsBoard para Python](#)
- [Documentación de PlatformIO](#)

Observación final

El repositorio documenta tanto una integración completa y funcional de lluvia y distancia como el desarrollo de alternativas de comunicación para escenarios de campo. La arquitectura XBee debe entenderse como una extensión de la plataforma ya construida: permite desacoplar el punto de medición del punto de procesamiento y preparar el sistema para estaciones remotas o con conectividad limitada.